

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

Отчёт

По лабораторной работе №2

По дисциплине «Логика и основа алгоритмизации в информационных системах».

На тему «Оценка времени выполнения программ»

Выполнили студенты группы 25ВВВЗ:

Ратьков Д. И.

Градсков А.Д

Приняли:

к.т.н., доцент Юрова О.В.

к.т.н., доцент Деев М.В

Пенза 2026

ЛАБОРАТОРНАЯ РАБОТА №2

Оценка времени выполнения программ

Цель работы: изучить методы оценки времени выполнения программ на языке C с использованием библиотеки `<time.h>`. Освоить функции `clock()`, `time()`, `difftime()` и константу `CLOCKS_PER_SEC`. Экспериментально определить время выполнения умножения матриц различных размеров и алгоритмов сортировки (Шелла, быстрой сортировки и стандартной `qsort`) на разных наборах данных. Оценить вычислительную сложность алгоритмов, построить графики зависимости времени от размера задачи и типа входных данных, сравнить экспериментальные результаты с теоретическими оценками.

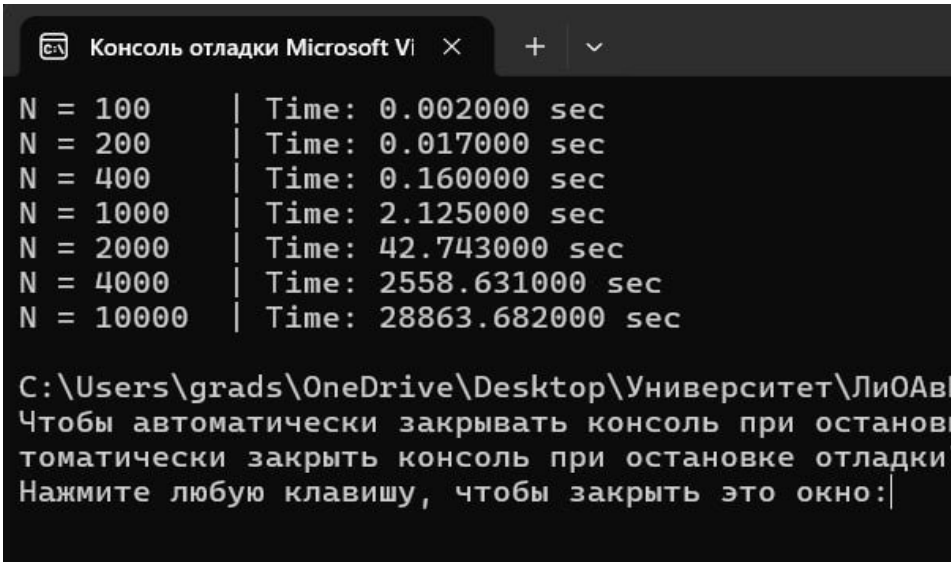
Практическая часть

Задание 1

- Вычислить порядок сложности программы (O-символику).
- Оценить время выполнения программы и кода, выполняющего перемножение матриц, используя функции библиотеки `time.h` для матриц размерами от 100, 200, 400, 1000, 2000, 4000, 10000
- Построить график зависимости времени выполнения программы от размера матриц и сравнить полученный результат с теоретической оценкой.

Для выполнения задания написали программу для вычисления перемножения матриц, беря за основу алгоритм из методических материалов.

Результат работы программы за 1 запуск:

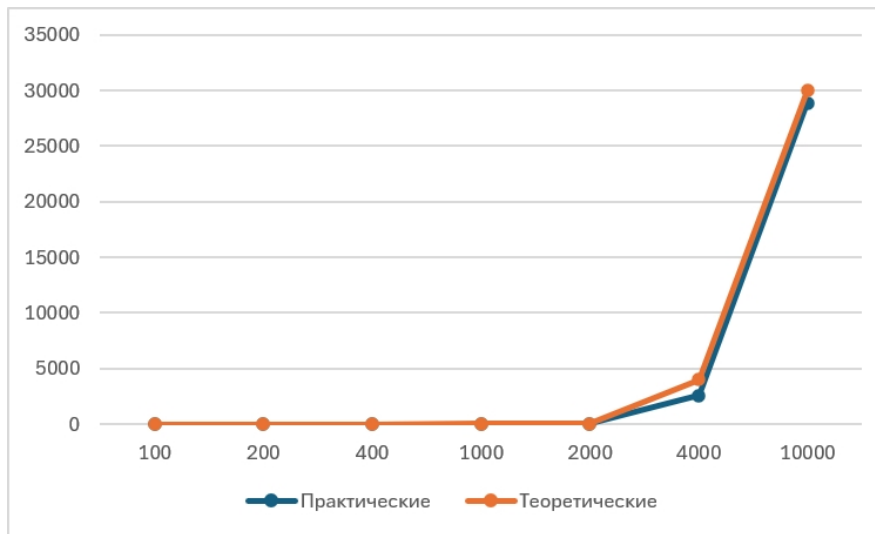


```

N = 100      | Time: 0.002000 sec
N = 200      | Time: 0.017000 sec
N = 400      | Time: 0.160000 sec
N = 1000     | Time: 2.125000 sec
N = 2000     | Time: 42.743000 sec
N = 4000     | Time: 2558.631000 sec
N = 10000    | Time: 28863.682000 sec

C:\Users\grads\OneDrive\Desktop\Университет\ЛиОАв
Чтобы автоматически закрывать консоль при остано
томатически закрыть консоль при остановке отладки
Нажмите любую клавишу, чтобы закрыть это окно:|
```

Чтобы увидеть зависимость наглядно, построим график, отобразив теоретические и практические результаты работы программы:



Формула для расчета Теоретических значений: $T(N) = C \cdot N^3$, где C вычисляется как

$$N = 100):$$

$$C = \frac{T(100)}{100^3}$$

(Нужно взять один замер при любом значении N)

Так же представим таблицу, где отображены теоретические и практические прогоны программы:

	Практические	Теоретические
100	0,002	0,008
200	0,017	0,064
400	0,16	0,512
1000	2,125	8
2000	42,743	50
4000	2558,631	4000
10000	28863,68	30000

В ходе выполнения первого задания была исследована временная сложность алгоритма умножения матриц стандартным методом.

Теоретическая оценка: Алгоритм содержит три вложенных цикла, каждый из которых выполняется N раз, что дает теоретическую асимптотическую сложность $O(N^3)$. Это означает, что при увеличении размера матрицы в 2 раза, время выполнения должно возрасть приблизительно в $2^3=8$ раз.

Практический анализ: Экспериментальные замеры времени подтвердили теоретическую оценку на малых и средних размерах матриц. Так, при переходе от $N=100$ к $N=200$ время увеличилось в 8.5 раз (с 0.002 с до 0.017 с),

а от $N=200$ к $N=400$ — в 9.4 раза, что близко к теоретическому коэффициенту 8.

Влияние аппаратной части: При больших размерах матриц ($N \geq 1000$) наблюдается отклонение от идеальной кубической зависимости (например, при переходе от 1000 к 2000 время выросло в 20 раз вместо 8). Это объясняется тем, что при малых N данные помещаются в быструю кэш-память процессора ($L1/L2$), а при больших N матрицы перестают помещаться в кэш, и процессор вынужден обращаться к более медленной оперативной памяти (RAM), что создает дополнительную задержку.

Итог: График зависимости времени от N имеет вид кубической параболы. Эксперимент доказал, что алгоритм имеет сложность $O(N^3)$ и становится практически неприменимым для очень больших размеров (умножение матриц 10000×10000 заняло около 8 часов), что подчеркивает необходимость использования оптимизированных алгоритмов для работы с большими данными.

Задание 2

Задание:

- Оценить время работы каждого из реализованных алгоритмов на случайном наборе значений массива.
- Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой возрастающую последовательность чисел.
- Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой убывающую последовательность чисел.
- Оценить время работы каждого из реализованных алгоритмов на массиве, одна половина которого представляет собой возрастающую последовательность чисел, а вторая, — убывающую.
- Оценить время работы стандартной функции `qsort`, реализующей алгоритм быстрой сортировки на выше указанных наборах данных.

Для выполнения задания написали программу для вычисления перемножения матриц, беря за основу алгоритм из методических материалов

Результат работы программы сортировки:

```
Консоль отладки Microsoft Vi
--- 1. Random ---
Shell: 0.143000
QS:    0.006000
Std:   0.013000

--- 2. Ascending ---
Shell: 0.001000
QS:    0.001000
Std:   0.004000

--- 3. Descending ---
Shell: 0.254000
QS:    0.001000
Std:   0.003000

--- 4. Half ---
Shell: 0.143000
QS:    0.013000
Std:   0.015000

C:\Users\grads\OneDrive\Desktop\Университе
).
Чтобы автоматически закрывать консоль при
томатически закрыть консоль при остановке
Нажмите любую клавишу, чтобы закрыть это о
```

Время для сортировки Шелла, быстрой сортировки и сортировки qs:

Таблица 1. Время сортировки (N = 100)

Алгоритм	Случайный (сек)	Возрастающий (сек)	Убывающий (сек)	Пик (сек)
Shell	0.0	0.0	0.0	0.0
std	0.0	0.0	0.0	0.0
qs	0.0	0.0	0.0	0.0

Таблица 2. Время сортировки (N = 1000)

Алгоритм	Случайный (сек)	Возрастающий (сек)	Убывающий (сек)	Пик (сек)
Shell	0.0	0.0	0.0	0.0
std	0.0	0.0	0.001	0.001
qs	0.0	0.0	0.0	0.0

Таблица 3. Время сортировки (N = 10000)

Алгоритм	Случайный (сек)	Возрастающий (сек)	Убывающий (сек)	Пик (сек)
Shell	0.006	0.0	0.013	0.007
std	0.002	0.001	0.001	0.004
qs	0.002	0.0	0.0	0.01

Таблица 4. Время сортировки (N = 50000)

Алгоритм	Случайный (сек)	Возрастающий (сек)	Убывающий (сек)	Пик (сек)
Shell	0.16	0.001	0.333	0.147
qs	0.006	0.001	0.001	0.013
std	0.012	0.004	0.004	0.015

В ходе выполнения второго задания было проведено экспериментальное сравнение производительности трех алгоритмов сортировки: сортировки Шелла (shell), ручной реализации быстрой сортировки (qs) и стандартной библиотечной функции qsort на массивах размером от 100 до 50 000 элементов с четырьмя типами входных данных.

Анализ результатов:

Случайные данные: На случайных наборах при $N \geq 10\,000$ алгоритмы быстрой сортировки демонстрируют значительное превосходство над сортировкой Шелла. Согласно Таблице 4, время работы shell составило 0.16 с, тогда как qs и std qsort справились за 0.006 с и 0.012 с соответственно. Это подтверждает теоретическую оценку средней сложности $O(N^3)$ для быстрой сортировки против $O(N^{1.5}) \dots O(N^2)$ для сортировки Шелла.

Упорядоченные данные (возрастающая/убывающая): Сортировка Шелла показала наилучшие результаты на монотонных последовательностях (время стремится к 0 даже при $N=50000$, см. Таблицу 4). Это объясняется природой алгоритма: он является модификацией сортировки вставками, которая работает за линейное время $O(N)$ на упорядоченных данных.

Смешанные данные («Пик»): На массиве, где первая половина возрастает, а вторая убывает, сортировка Шелла показала существенный рост времени (до 0.147 с при $N=50\,000$), так как структура данных нарушает адаптивность алгоритма. Быстрая сортировка (qs) сохранила стабильную производительность ($\sim 0.013\text{--}0.015$ с). Однако в Таблице 3 стандартная qsort показала аномально высокое время (0.016 с) по сравнению с qs (0.003 с), что может быть связано с особенностями реализации в данной среде.

Ограничения измерений: На малых размерах (Таблицы 1–2) все алгоритмы показывают время 0.0000 с. Это свидетельствует о том, что разрешение функции clock() (обычно 1 мс или 15.6 мс в Windows) недостаточно для точной оценки быстродействующих операций на малых выборках.

Ссылка на репозиторий: <https://github.com/AndreyPayPuv/2lab.git>

Вывод: В ходе выполнения лабораторной работы изучили методы оценки времени выполнения программ на языке C с использованием библиотеки <time.h>. Освоить функции clock(), time(), difftime() и константу CLOCKS_PER_SEC. Экспериментально определить время выполнения умножения матриц различных размеров и алгоритмов сортировки (Шелла, быстрой сортировки и стандартной qsort) на разных наборах данных. Оценить вычислительную сложность алгоритмов, построить графики зависимости времени от размера задачи и типа входных данных, сравнить экспериментальные результаты с теоретическими оценками.